

How to Compile and Run this JNI Program

This guide provides the necessary steps to compile and run the Java Native Interface (JNI) example.

Prerequisites:

1. **JDK (Java Development Kit):** You need `javac` (compiler) and `java` (runtime).
2. **C Compiler:** You need `gcc` (or a similar compiler like `clang`). On Windows, you can use MinGW or WSL (Windows Subsystem for Linux).
3. **JAVA_HOME Environment Variable:** This must be set and pointing to your JDK installation directory.
 - **macOS:** `/Library/Java/JavaVirtualMachines/jdk-XX.X.X.jdk/Contents/Home`
 - **Linux:** `/usr/lib/jvm/java-XX-openjdk-amd64`
 - **Windows:** `C:\Program Files\Java\jdk-XX.X.X`

Step-by-Step Instructions

Follow these steps precisely in your terminal or command prompt.

Step 1: Save the Files

Save `ArithmeticNative.java` and `ArithmeticNative.c` in a new, empty directory.

Step 2: Compile the Java Code

This creates the `.class` file.

```
javac ArithmeticNative.java
```

Step 3: Generate the C Header File

This is the most important step. It generates `ArithmeticNative.h`, which creates the correct function signatures for your C code to use.

```
javac -h . ArithmeticNative.java
```

After this, you will see a new file named `ArithmeticNative.h` in your directory.

Step 4: Compile the C Code into a Shared Library

This command compiles your `ArithmeticNative.c` file and links it against the JNI headers provided by your JDK.

This command is specific to your Operating System.

- **For Linux:**
`gcc -shared -o libarithmetric.so -I"$JAVA_HOME/include" -I"$JAVA_HOME/include/linux" -I. ArithmeticNative.c`
- **For macOS:**
`gcc -shared -o libarithmetric.dylib -I"$JAVA_HOME/include" -I"$JAVA_HOME/include/darwin" -I. ArithmeticNative.c`
- **For Windows (using MinGW):**
`gcc -shared -o arithmetric.dll -I"%JAVA_HOME%\include" -I"%JAVA_HOME%\include\win32" -I. ArithmeticNative.c`

This command creates the actual native library file (`libarithmetric.so`, `libarithmetric.dylib`, or `arithmetric.dll`) that Java will load.

Step 5: Run the Java Program

Finally, run the Java program. You must use the `-Djava.library.path=.` flag to tell Java to look for the native library in the *current directory* (which is what `.` means).

```
java -Djava.library.path=. ArithmeticNative
```

Example Output:

You should see an output similar to this:

```
--- Calling C functions from Java ---  
Running on: Linux
```

```
Addition:      100 + 20 = 120  
Subtraction:    100 - 20 = 80  
Multiplication: 100 * 20 = 2000  
Division:       100 / 20 = 5.00
```

```
--- Testing Division by Zero ---  
C Error: Division by zero!  
Division:       10 / 0 = 0.00
```